

UD 4. APLICACIÓN DE LAS ESTRUCTURAS DE ALMACENAMIENTO

**Programación – 1º DAM
Curso 2025/2026**

CONTENIDOS

- Introducción
- Pseudocódigo
- Propiedades
- Vectores – Arrays unidimensionales
- Matrices – Arrays bidimensionales
- Clase Arrays
- Clase String

Introducción

- Las estructuras de datos son elementos fundamentales en la programación que **permiten almacenar y organizar datos de una forma eficiente**. En Java, las estructuras de datos son utilizadas para almacenar, manipular y acceder a información de diversas formas.
- La elección de una determinada estructura de datos permite a los programadores gestionar datos de manera eficaz, así como optimizar la memoria y recursos del servidor o PC en la que es ejecutada.
- Una primera clasificación en las estructuras de almacenamiento podría ser: estructuras de almacenamiento **estáticas** y estructuras de almacenamiento **dinámicas**.

Introducción

- Estructuras de almacenamiento **estáticas**
 - El tamaño ocupado en la memoria **se define antes de que el programa se ejecute y no puede modificarse durante la ejecución del programa**. Están implementadas en casi todos los lenguajes y son los registros, ficheros y matrices. Dependiendo del tipo de dato pueden ser:
 - Homogéneas: todos los subelementos son del mismo tipo: arrays o matrices.
 - Heterogéneas: los subelementos contienen datos de distintos tipos: registros.

Introducción

- Estructuras de almacenamiento **dinámicas**
 - El **tamaño** de la **estructura se va modificando** conforme avanza el programa; es decir, su tamaño cambia en tiempo de ejecución. Pueden ser:
 - Lineales: listas, pilas, colas.
 - No lineales: árboles y grafos.

Introducción

- Un **array** o **vector** es una colección de valores de un mismo tipo dentro de una misma variable. De forma que se puede acceder a cada valor independientemente.
- Para Java, además, un array es un objeto que tiene propiedades que se pueden manipular.
- Los arrays solucionan problemas concernientes al manejo de muchas variables que se refieren a datos similares.
- Gracias a los arrays se puede crear un conjunto de variables con el mismo nombre. La diferencia será que un número (índice del array) distinguirá a cada variable.

Arrays en Pseudocódigo

- Un array, también llamado matriz, arreglo o formación de elementos (datos), es un tipo de estructura que agrupa, bajo el mismo nombre de variable, a una colección de elementos (datos) del mismo tipo.
- Dicha colección de elementos puede ser de una o varias dimensiones:
 - Arrays unidimensionales
 - Arrays multidimensionales

Arrays en Pseudocódigo

ARRAYS UNIDIMENSIONALES

- A los arrays unidimensionales también se les denomina **vectores** y, en pseudocódigo, se pueden declarar utilizando la siguiente sintaxis:

Dimension *<nombre_del_array>*[*<tamaño>*]

El **<tamaño>** del array determina el número de valores (de un mismo tipo de dato) que pueden ser almacenados en el array.

- Para indicar el **tipo de dato** de los valores del array, se puede utilizar la siguiente sintaxis:

Definir *<nombre_del_array>* **Como** *<tipo_de_dato>*

Arrays en Pseudocódigo

ACCIONES SOBRE UN ARRAY

- Sobre un array, o mejor dicho, sobre los elementos de un array, se pueden realizar las mismas acciones (asignación, salida y entrada) que se podrían hacer sobre una variable del mismo tipo que los elementos de dicho array y, además, se pueden llevar a cabo otras acciones (recorrido, ordenación, búsqueda...).

Arrays en Pseudocódigo

Ejemplo: definimos un array de 4 elementos de tipo Entero, asignándole un valor a cada uno de ellos y lo recorremos, visualizando todos ellos por pantalla:

// Ejemplo: Recorrido de un array

Algoritmo Recorrido_de_un_Array

// Declaración de un array de 4 elementos

Dimension datos[4];

Definir datos, indice **Como** Entero;

// Asignación de valores a los elementos del array

datos[1] = 5;

datos[2] = 1;

datos[3] = 8;

datos[4] = 2;

// Recorrido del array mostrando por pantalla los valores de cada elemento

Para indice =1 **Hasta** 4 **Con Paso** 1 **Hacer**

..... **Escribir** datos[indice];

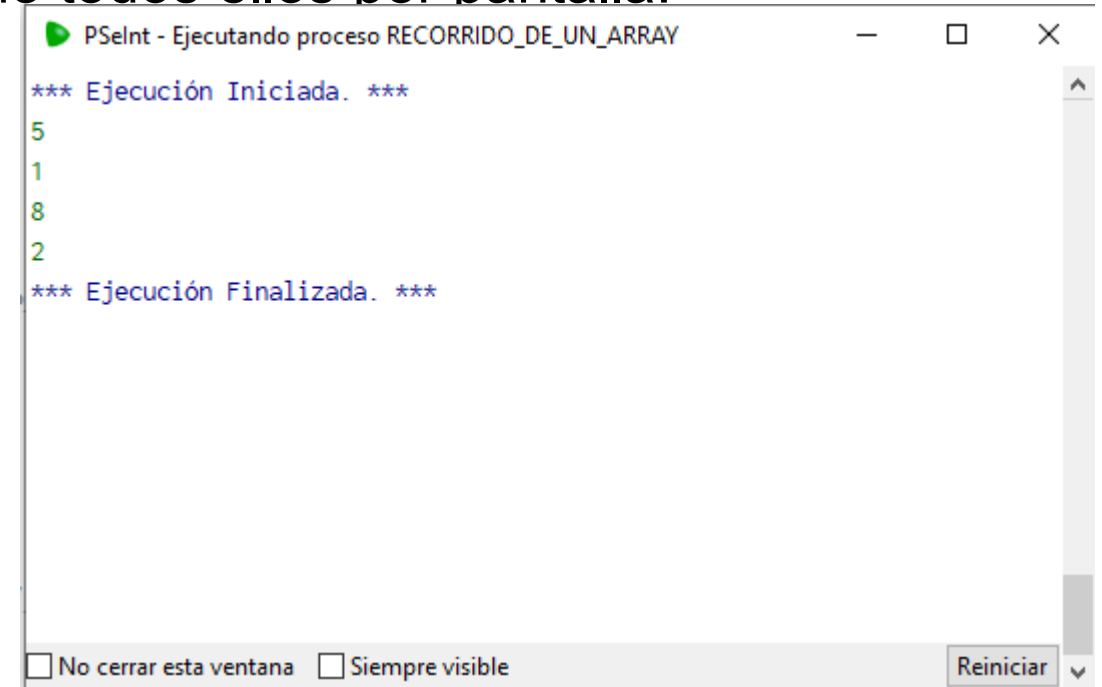
FinPara

FinAlgoritmo

Arrays en Pseudocódigo

Ejemplo: definimos un array de 4 elementos de tipo Entero, asignándole un valor a cada uno de ellos y lo recorremos, visualizando todos ellos por pantalla:

```
1  // Ejemplo: Recorrido de un array
2  Algoritmo Recorrido_de_un_Array
3
4      // Declaración de un array de 4 elementos
5      Dimension datos[4];
6
7      Definir datos, indice Como Entero;
8
9      // Asignación de valores a los elementos del array
10     datos[0] = 5;
11     datos[1] = 1;
12     datos[2] = 8;
13     datos[3] = 2;
14
15     // Recorrido del array mostrando por pantalla los valores de cada elemento
16     Para indice =1 Hasta 4 Con Paso 1 Hacer
17         Escribir datos[indice];
18     FinPara
19
20 FinAlgoritmo
```



```
PSeInt - Ejecutando proceso RECORRIDO_DE_UN_ARRAY
*** Ejecución Iniciada. ***
5
1
8
2
*** Ejecución Finalizada. ***
☐ No cerrar esta ventana ☐ Siempre visible 
```

Arrays en Pseudocódigo

Si nos fijamos en el ejemplo anterior, a la hora de asignar valores, se ha empezado en el índice 0. La razón es que la mayoría de los lenguajes de programación el índice puede oscilar entre 0 y $n-1$. Ejemplo de lenguaje que trabaja de esta forma es Java, lenguaje que trabajaremos a lo largo del curso.

Arrays en Pseudocódigo

ARRAYS MULTIDIMENSIONALES

- A los arrays multidimensionales también se les denomina **matrices** y, en pseudocódigo, se pueden declarar utilizando la siguiente sintaxis:

Dimension *<nombre_del_array>*[*<tamaño_1>*, ..., *<tamaño_n>*]

El **<tamaño>** del array determina el número de valores (de un mismo tipo de dato) que pueden ser almacenados en cada una de sus dimensiones.

- Para indicar el **tipo de dato** de los valores del array, se puede utilizar la siguiente sintaxis:

Definir *<nombre_del_array>* **Como** *<tipo_de_dato>*

Arrays en Pseudocódigo

Ejemplo: Se quiere diseñar el algoritmo de un programa que:

- 1º) Cree una matriz (de 2 filas por 3 columnas, 2x3) para almacenar números enteros.
- 2º) Muestre un menú con 4 opciones:
 1. Mostrar los números de la matriz.
 2. Asignar números por teclado a la matriz.
 3. Inicializar la matriz a ceros.
 4. Salir.
- 3º) Pida por teclado la opción deseada (dato carácter). Deberá ser introducida, hasta que, sea mayor o igual que '1' y menor o igual que '4'.
- 4º) Ejecute la opción del menú seleccionada.
- 5º) Repita los pasos 1º, 2º y 3º, hasta que, el usuario seleccione la opción 4 (Salir) del menú.

Ver Ejemplo_Array_de_Dos_Dimensiones.psc

Propiedades

- Algunas propiedades de los arrays son:
 - Los arrays se utilizan como contenedores para almacenar datos relacionados (en lugar de declarar variables por separado para cada uno de los elementos del array).
 - **Todos los datos** incluidos en el array **son del mismo tipo**.
 - El **tamaño** del array **se establece cuando se crea el array**.
 - A los elementos del array **se accederá a través de la posición** que ocupan dentro del conjunto de elementos del array.
 - Los **arrays unidimensionales** se conocen con el nombre de **vectores**.
 - Los **arrays bidimensionales** se conocen con el nombre de **matrices**.

Vectores (Arrays unidimensionales)

Declaración

- Un array se declara de forma similar a una variable simple, pero añadiendo **corchetes** para indicar que se trata de un array y no de una variable simple del tipo especificado.

- Se puede declarar de dos formas:

tipo identificador[];

tipo[] identificador;

donde tipo es el tipo de dato de los elementos del vector e identificador es el nombre de la variable.

Vectores (Arrays unidimensionales)

- Ejemplos:

```
int notas[ ];    //variable llamada notas de tipo array de enteros
```

```
double cuentas[ ];//variable llamada cuentas de tipo array de decimales
```

Esta declaración indica para qué servirá el array, pero no reserva espacio en la memoria RAM al no saberse todavía el tamaño del mismo. Todavía no puede utilizarse el array, falta instanciarlo.

Vectores (Arrays unidimensionales)

Instancia (creación)

- Tras la declaración del array, se tiene que instanciar, para ello se utiliza el operador **new**, que es el que realmente crea el array indicando un tamaño. Cuando se usa new, es cuando se reserva el espacio necesario en memoria.

- Ejemplo:

```
int notas[]; // Declaramos 'notas' como array de tipo int
notas = new int[3]; // Instanciamos 'notas' a tamaño 3
// Es habitual declarar e instanciar en una sola línea
int notas[] = new int[3];
```

Vectores (Arrays unidimensionales)

Almacenamiento

- Los valores del array se asignan (almacenan) utilizando el índice del mismo entre corchetes.
- El **primer elemento del vector** siempre estará en la posición o **índice 0**.

Índices → 0 1 2 3 4

Valores →

10	15	32	28	9
----	----	----	----	---

- Por ejemplo, para almacenar el valor 8 en la tercera posición del array escribiríamos:

```
notas[2] = 2;
```

Vectores (Arrays unidimensionales)

- También se pueden asignar valores al array en la propia declaración e instanciación:

```
int notas[] = {8, 10, 2, 3, 5};
```

```
int notas2[] = new int[] {8, 10, 2, 3, 5}; //Equivalente a  
la anterior
```

Vectores (Arrays unidimensionales)

- Esto declara e inicializa un array de tres elementos. El ejemplo sería equivalente a:

```
notas[0] = 8;  
notas[1] = 10;  
notas[2] = 2;  
notas[3] = 3;  
notas[4] = 5;
```

- Se pueden declarar arrays a cualquier tipo de datos (enteros, booleanos, doubles, ... e incluso objetos).

Vectores (Arrays unidimensionales)

Longitud de un vector

- Los arrays poseen una propiedad llamada **length** que indica su tamaño.
- Ejemplo:

```
int notas[] = new int[4]; //Declara e instancia vector tipo int de  
tamaño 4
```

```
System.out.println( notas.length ); // Mostrará un 4
```

- Si el vector tiene como en el ejemplo 4 elementos, la propiedad length nos devolverá el valor entero 4, pero su primer elemento se encuentra en notas[0] y el último en notas[3].

Vectores (Arrays unidimensionales)

Recorrido de un vector (acceso a sus elementos)

- Para recorrer un vector siempre es necesario un bucle.
- Ejemplo: declaramos e instanciamos un vector tipo int con las notas de un alumno y luego usamos un bucle for para recorrer el vector y mostrar sus los elementos.

```
// Declaramos e instanciamos vector tipo int
int notas[] = new int[] {7, 3, 9, 6, 5};

// Como el vector es de tamaño 5 sus elementos estarán en las posiciones
de 0 a 4

// Recorremos el vector desde i=0 hasta i<5 (es decir, desde 0 hasta 4)
for (int i = 0; i < notas.length; i++) {
    System.out.println(notas[i]);
}
```

Vectores (Arrays unidimensionales)

Ahora vamos a calcular la nota media (sumar todas y luego dividir entre el número de notas):

```
// Declaramos suma y media
int suma = 0;
int media;

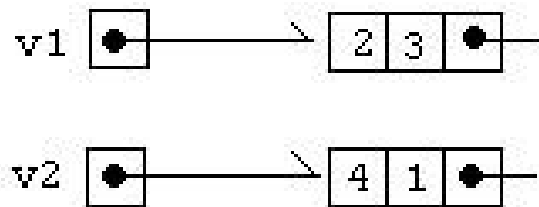
//Recorremos el vector desde 0 hasta 4, acumulando las notas,
en suma
for (int i = 0; i < notas.length; i++) {
    suma += notas[i]; //Equivale a: suma = suma + notas[i]
}

// Calculamos la media y la mostramos por pantalla
media = suma / notas.length;
System.out.println("La nota media es: " +
media);
```

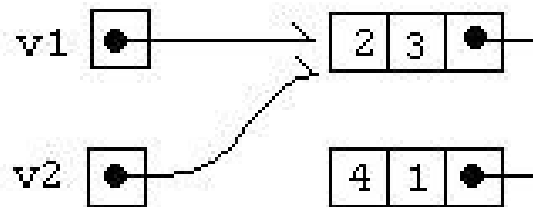

Vectores (Arrays unidimensionales)

Copia de vectores

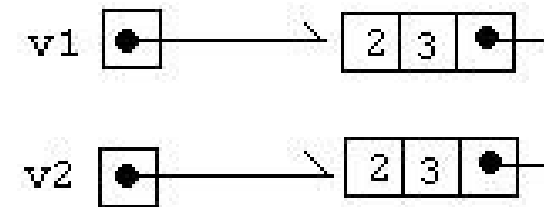
- Para copiar vectores **no basta con igualar un vector a otro como si fuera una variable simple.**
- Si partimos de dos vectores v1, v2 e hiciéramos $v2=v1$, lo que ocurriría sería que v2 apuntaría a la posición de memoria de v1. Eso es lo que se denomina copia de referencia:



Situación
Inicial



Copia de
referencia



Copia de
Contenido

Vectores (Arrays unidimensionales)

Copia de vectores

- La forma que vamos a usar para copiar arrays va a ser copiar sus elementos uno a uno:

```
for (i=0; i<v1.length; i++) {  
    v2[i]=v1[i];  
}
```

- También podría hacerse con la función arraycopy:

```
System.arraycopy(v_origen, i_origen, v_destino, i_destino, length);
```

v_origen: Vector origen

i_origen: Posición inicial de la copia

v_destino: Vector destino

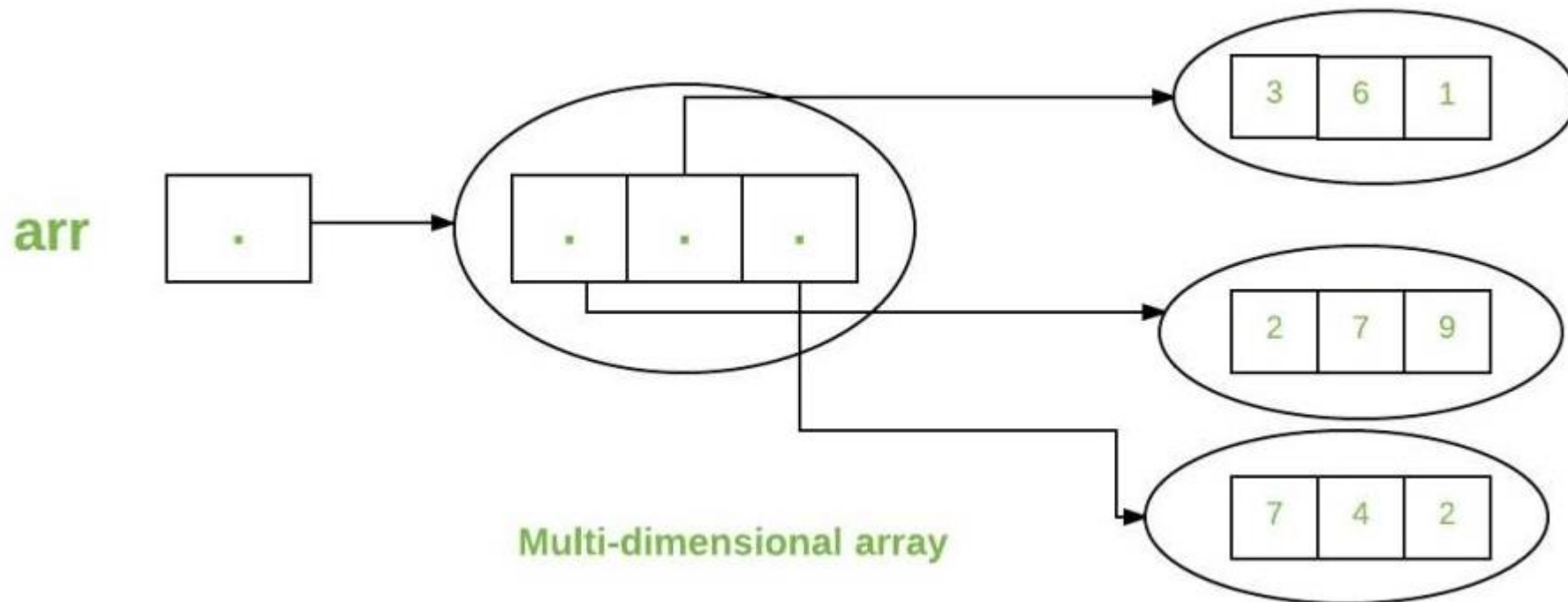
i_destino: Posición final de la copia copia

length: Cantidad de elementos a copiar

```
// Copiamos todos los elementos de v1 en v2  
System.arraycopy(v1, 0, v2, 0, v1.length);
```

Matrices (Arrays multidimensionales)

Los **arrays bidimensionales**, también llamados **matrices**, son arrays en dos dimensiones. En la práctica se puede interpretar como un array unidimensional en el que hay almacenado otro array en cada elemento del primero.



Matrices (Arrays multidimensionales)

Declaración e instancia de matrices

Se declaran e inicializan de la misma manera que los arrays unidimensionales, pero esta vez hay que tratar con dos dimensiones. Al poderse interpretar como una matriz, la primera dimensión hará referencia a las filas, y la segunda a las columnas.

tipo identificador[][];

tipo[][] identificador;

Matrices (Arrays multidimensionales)

Por ejemplo, declaramos e instanciamos un array de 3 x 3 (3 filas x 3 columnas).

```
double precios[][] = new double[3][3];
```

Accedemos a sus valores utilizando dobles corchetes.

```
precios[0][0] = 7.5;
```

```
precios[0][1] = 12;
```

```
precios[0][2] = 0.99;
```

```
precios[1][0] = 4.75;
```

```
// etc.
```

		COLUMNAS		
		0	1	2
FILAS	0	(0,0)	(0,1)	(0,2)
	1	(1,0)	(1,1)	(1,2)
	2	(2,0)	(2,1)	(2,2)

Matrices (Arrays multidimensionales)

Recorrido de matrices

Al tener varias dimensiones (a partir de ahora nos centramos en 2 dimensiones), tenemos que utilizar bucles anidados para su recorrido, ya que, por cada fila, tendremos que recorrer todas sus columnas.

Matrices (Arrays multidimensionales)

Por ejemplo, declaramos un array de 3x6 (3 filas por 6 columnas) para almacenar las notas de 3 alumnos (fila → alumno, columna → nota alumno)

```
int notas[ ][ ] = new int[3][6];  
// Para cada fila (alumno)  
for (int i = 0; i < notas.length; i++) {  
    System.out.print("Notas del alumno " + i + ": ");  
    // Para cada columna (nota)  
    for (int j = 0; j < notas[i].length; j++) {  
        System.out.print(notas[i][j] + " ");  
    }  
}
```

Clase Arrays

En Java disponemos de la clase **Arrays** que facilita algunas de las operaciones más comunes que se pueden llevar a cabo con los arrays:

- **Arrays.binarySearch()** Busca en un array un elemento que se pasa como parámetro. Devolverá la posición donde lo encuentra o el valor -1 si no existe.

Hay que tener en cuenta que el array deberá de estar ordenado.

```
int[] numeros = new int[10];
for (int i = 0; i < numeros.length; i++) {
    numeros[i] = i * 10;
}
int resultado = Arrays.binarySearch(numeros, 20);
System.out.println(resultado); // 2
```


Clase Arrays

- **Arrays.copyOf()** Permite copiar el array a otro de diferente longitud (mayor o menor), rellenando o truncando los valores nuevos o eliminados, respectivamente. Devuelve el array resultante

```
int[] numeros = new int[5];  
  
int[] masNumeros = Arrays.copyOf(numeros, 10);  
  
System.out.println(masNumeros.length); // 10
```

- **Arrays.equals()** Comprueba si dos arrays son iguales. Devuelve un valor booleano indicando si son o no iguales

```
int[] numeros = {1, 3, 4};  
  
int[] otrosNumeros = {1, 3, 4};  
  
System.out.println(Arrays.equals(numeros, otrosNumeros));  
  
true
```

Clase Arrays

- **Arrays.fill()** Rellena un array con el valor que se le pasa como parámetro

```
int[] numeros = new int[3];  
  
Arrays.fill(numeros, 5);  
  
System.out.println(Arrays.toString(numeros));    //[5,5,5]
```

- **Arrays.sort()** Ordena el array que se pasa como parámetro

```
int[] numeros = {4, 10, 3, 6}  
  
Arrays.sort(numeros);  
  
System.out.println(Arrays.toString(numeros));    //[3,4,6,10]
```

- **Arrays.toString()** Devuelve una representación en String del array que se pasa como parámetro

```
int[] numeros = {10, 20, 30, 5}  
  
System.out.println(Arrays.toString(numeros));    //[10,20,30,5]
```

Clase String

Java Docs

Java además de un lenguaje de programación y una plataforma sobre la que ejecutar nuestros programas, es un gran almacén de código para utilizar cuando estemos programando.

Java SE (Standard Edition) pone a nuestra disposición más de 4000 clases con sus correspondientes métodos para utilizar cuando necesitemos.

¿Dónde puedo yo ver y consultar todas esas clases?

<https://docs.oracle.com/javase/8/docs/api/>

Clase String

Una cadena (**String**) es una secuencia de caracteres, incluidos **letras** del alfabeto, **números**, **caracteres especiales** y **espacios** en blanco.

Java no tiene para la **manipulación de cadenas** un tipo primitivo de variable como float, double, etc..., para ello utiliza la clase **String**.

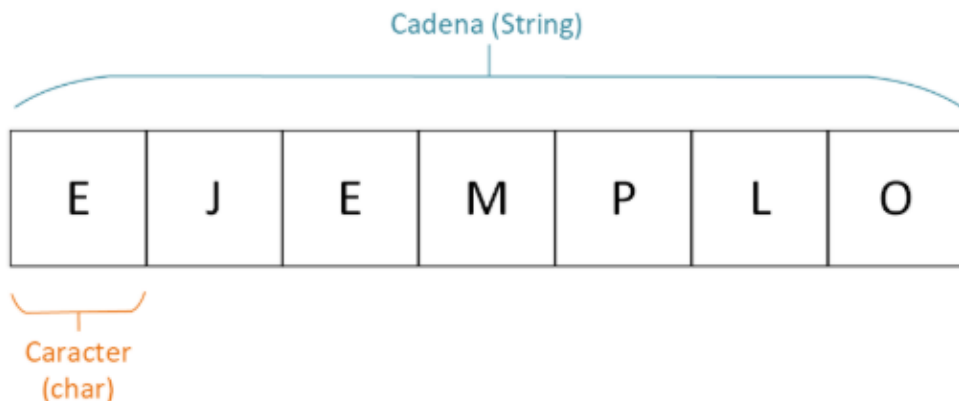
A todos los efectos una variable de tipo **String** es un **objeto** de la clase **java.lang.String**

Para asignar un **char** utilizaremos comillas simples:

```
char letra='j';
```

Para asignar un **String** utilizaremos comillas dobles:

```
String nombre="Pilar";
```



Clase String

Declaración

La sintaxis para declarar una cadena de texto es:

```
String <nombre_cadena>;
```

Podemos crear una cadena de texto de 3 maneras distintas:

- `String cadena1=new String("Este es el valor que se asigna a la variable cadena");`
- `String cadena1="Hola";`
`String cadena2=new String(cadena1); //asignamos a la cadena2 el valor de la variable cadena1`
- `String cadena3="Este es el valor que se asigna a la variable";`

Clase String

Características

- La clase String está **indexada**, esto significa que cada uno de los caracteres tiene asociado un índice: 0 para el primero, 1 para el segundo, etc., por tanto, **podremos acceder a un carácter indicando su posición**.

H	O	L	A		P	...
0	1	2	3	4	5	...

- Podemos crear una **cadena vacía** de dos maneras:

```
String cadena="";
```

```
String cadena=new String();
```

- Podemos utilizar el operador + para concatenar:

```
String alumno="Pilar"+" García";
```

El valor de alumno es: "Pilar García"

Clase String. Métodos para obtener tamaño y subcadenas

length()

int length(): devuelve un entero con la **cantidad de caracteres** que tiene la cadena:

```
String str="DAW";
```

```
int longitud=str.length(); //longitud tendrá el valor 3
```

```
String str="El primer programa";
```

```
int longitud=str.length(); //longitud tendrá el valor 18
```

Clase String. Métodos para obtener tamaño y subcadenas

substring()

String substring(int pos): extrae y devuelve una cadena desde la posición *pos* inclusive hasta el final de la cadena:

```
String frase = "Esta es mi casa";
```

```
String frase2 = frase.substring(5); //frase2 valdrá "es mi casa"
```

String substring(int indiceInicial, int indiceFinal): devuelve una cadena con el segmento de la cadena original entre la posición **indiceInicial** inclusive hasta una posición anterior a **indiceFinal**:

```
String str="Buenos días";
```

```
System.out.println(str.substring(0,6)); //Desde la posición 0
```

```
hasta la 5: se mostrará por pantalla Buenos
```


Clase String. Métodos para comparar cadenas

equals()

boolean equals (String cadEnv): Compara el contenido del **String** con el contenido de la cadena enviada, devuelve **true si son iguales**. **Distingue entre mayúsculas y minúsculas.**

```
String cadena1 = "Buenos días";
String cadena2 = "Buenas tardes";
if(cadena1.equals(cadena2)) {
    System.out.println("Las cadenas son iguales");
}else{
    System.out.println("Las cadenas no son iguales");
} //Por pantalla se mostrará que las cadenas no son iguales
```

Clase String. Métodos para comparar cadenas

`equalsIgnoreCase()`

boolean equalsIgnoreCase(String cadEnv): Compara el **String** con la cadena enviada **ignorando mayúsculas y minúsculas**, devuelve **true si son iguales**.

```
String cadena1 = "Hola";
String cadena2 = "hola";
if(cadena1.equalsIgnoreCase(cadena2)){
    System.out.println("Las cadenas son iguales");
}
else{
    System.out.println("Las cadenas no son iguales");
} //Por pantalla se mostrará que las cadenas son iguales
```

Clase String. Métodos para comparar cadenas

No es lo mismo comparar cadenas con == que con equals()

El operador == se utiliza para verificar si dos objetos son iguales.

Mientras que con **equals()** comparamos el valor de los objetos, por tanto, cuando queramos comprobar si dos cadenas tienen el mismo valor debemos usar equals().

```
String str1 = new String("El lenguaje Java");
String str2 = new String("El lenguaje Java");
if (str1 == str2)
    System.out.println("Los mismos objetos con distinto nombre");
else
    System.out.println("Distintos objetos con distinto nombre");

if (str1.equals(str2))
    System.out.println("el mismo contenido");
else
    System.out.println("distinto contenido");
```

Este código nos devolverá que tenemos distintos objetos con distinto nombre, pero con el mismo contenido.

Clase String. Métodos para comparar cadenas

compareTo()

int compareTo(String cadEnv): compara lexicográficamente el contenido del String con el de la cadena enviada, usando el **valor Unicode** de cada carácter.

Devuelve:

- un 0 si son iguales,
- un número negativo si el String va antes (alfabéticamente) que la cadEnv
- un número mayor que 0 si va después.

```
String a = "amar";  
String b = "querer";  
int valor1 = a.compareTo(b); // -1 , "amar" va antes que "querer"  
int valor2 = b.compareTo(a); // 1, "querer" va después que "amar"  
int valor3 = a.compareTo("amar"); // 0  
System.out.println(valor1);  
System.out.println(valor2);  
System.out.println(valor3);
```

Clase String. Métodos para comparar cadenas

El método `compareTo()` distingue mayúsculas y minúsculas. Si no queremos tenerlas en cuenta usaremos:

`int compareToIgnoreCase(String cadena)`

Clase String. Métodos para buscar en una cadena

charAt()

char charAt (int pos) : devuelve el **carácter** que está en la posición **pos**.

```
String cadena = "Buenos días";  
char character = cadena.charAt(1); // El caracter que ocupa la posición 1 en la variable cadena es 'u'
```

indexOf()

int indexOf(String cadEnv): busca en la cadena de invocación la subcadena enviada cadEnv. Devuelve el índice de la 1ª coincidencia o -1 en caso de no encontrarla.

```
String str = "Buenos días";  
//Buscamos la 1ª posición de un caracter en la cadena str  
int pos1 = str.indexOf('s'); // La primera posición en la que se encuentra la 's' es 5  
int pos2 = str.indexOf("nos"); // La primera posición en la que se encuentra la cadena "no" es 3  
int pos3 = str.indexOf("b"); // No se encuentra la b, devolverá -1  
// A partir de una posición:  
int pos4 = str.indexOf('s', 6); // A partir de la posición 6, la primera vez que se encuentra la 's' es en la posición 10
```

Clase String. Métodos para buscar en una cadena

lastIndexOf()

int lastIndexOf(String cadEnv): busca en la cadena de invocación la subcadena enviada (cadEnv) y **devuelve el índice de la última coincidencia** o -1 en caso de no encontrarlo.

```
String str = "Buenos días en este día";  
int pos1 = str.lastIndexOf('s'); // La última aparición del carácter 's' es en la posición 16  
int pos2 = str.lastIndexOf("día"); // La última aparición de la cadena "día" está en la posición 20  
int pos3 = str.lastIndexOf("hola"); // No se encuentra "hola", devolverá -1  
//A partir de una posición hacia atrás:  
int pos4 = str.lastIndexOf('s', 6); // La última aparición de la 's' hasta la posición 6 es 5
```

Clase String. Métodos para modificar una cadena

toLowerCase()

String toLowerCase(): devuelve el String convertido a **minúsculas**.

```
String cadena = "MoNtaÑaRuSa";  
String cadena2 = cadena.toLowerCase();//Convierte la cadena a: montañarusa
```

toUpperCase()

String toUpperCase(): devuelve el String convertido a **mayúsculas**.

```
String cadena = "MoNtaÑaRuSa";  
String cadena2 = cadena.toUpperCase();//Convierte la cadena a: MONTAÑARUSA
```

trim()

String trim(): Elimina espacios en blanco al principio y al final de la cadena. No elimina los espacios situados entre las palabras.

```
String str="      Buenos días tengamos      ";  
String nuevoStr=str.trim();//Buenos días tengamos
```


Clase String. Métodos para modificar una cadena

replace()

String replace(char cAnterior, char cNuevo): Devuelve la cadena original **cambiando** los **caracteres** iguales a cAnterior por cNuevo.

```
String str="Buenos días tengamos";  
String nuevoStr=str.replace("os", "OS"); //Convierte la cadena a BuenOS días tengamOS
```

replaceFirst()

String replaceFirst(String str1, String str2): Devuelve la cadena original pero **cambiando** primera aparición de la cadena **str1** por la cadena **str2**.

```
String str="Buenos días tengamos";  
String nuevoStr=str.replaceFirst("os", "OS"); //Convierte la cadena a BuenOS días tengamos
```

Clase String. Métodos para modificar una cadena

split()

String[] split (String regex) Devuelve un array de String resultado de partir la cadena usando como separador el argumento (cadena o expresión regular).

```
String cadena="Hola, cómo estás.";
String resul[];

resul=cadena.split(" ");
for(int i=0;i<resul.length;i++) {
    System.out.println(resul[i]);    //Hola,
                                     //cómo
                                     //estás.
```

Clase String. Concatenar cadenas

La concatenación de cadenas en Java es una operación para unir dos o más cadenas y devolver una nueva.

Con el operador +

El operador + permite concatenar una cadena con otra.

```
String frase1 = "Esta es mi casa";  
String frase2 = " de vacaciones";  
String fraseCompleta=frase1+frase2;//Esta es mi casa de vacaciones
```

¡¡Atención!!

Si uno de los operandos es una cadena, Java convierte automáticamente los tipos de dato que no son cadenas en cadena antes de la concatenación:

```
String str="Aprendiendo Java " +8;  
System.out.println(str); //Aprendiendo Java 8  
  
String NumStr= "8"+8;  
System.out.println(NumStr); //88
```

Clase String. Concatenar cadenas

Con el método `concat ()`

String concat(String cadena2): agrega la cadena especificada (cadena2) al final de la cadena dada y devuelve una nueva cadena.

```
String frase = "Esta es mi casa";
```

```
String frase2 = frase.concat(" de vacaciones");// Esta es mi casa de vacaciones
```

```
String frase3 = " de muñecas";
```

```
String frase4 = frase.concat(frase3);// Esta es mi casa de muñecas
```

Clase String. Formatear cadenas

En Java podemos dar a las cadenas un formato predefinido utilizando el método **format()** al que le pasaremos los siguientes parámetros:

- **Cadena de formato:** esta cadena especifica cómo será el formato de salida. En ella se mezclará texto normal con los *especificadores de formato* que indicarán cómo se deben formatear los datos.
- **Lista de argumentos:** son variables que contienen los datos que se formatearán. Tiene que haber tantos argumentos como especificadores de formato haya en la cadena de formato.

Clase String. Formatear cadenas

Los **especificadores de formato** comienzan siempre por "%", es lo que se denomina un carácter de escape (carácter que sirve para indicar que lo que hay a continuación no es texto normal, sino algo especial). El especificador de formato debe llevar como mínimo el símbolo "%" y un carácter que indica la conversión a realizar.

La conversión se indica con un carácter e indica al método format cómo debe ser formateado el argumento. Dependiendo del tipo de dato podemos usar unas conversiones u otras. Veamos las conversiones más utilizadas:

Clase String. Formatear cadenas

%s , %S Formato: String

En el siguiente código se sustituirían los especificadores %s y %S por el valor de las variables nombre y apellido respectivamente:

```
String nombre="Cristina";  
String apellidos="Castellanos";  
  
String nombreCompleto=String.format("Hola, me llamo %s %S", nombre, apellidos);  
System.out.println(nombreCompleto); // Hola, me llamo Cristina CASTELLANOS
```

%d Formato: número entero

El especificador %d se usa para formatear números enteros (*int*, *long*).

```
int cantidad=3;  
String total=String.format("Cantidad: %d cafés",cantidad);  
System.out.println(total); //Cantidad: 3 cafés
```

Clase String. Formatear cadenas

%f Formato: número decimal

El especificador **%f** se usa para formatear números decimales (*float*, *double*). Por defecto, muestra 6 cifras decimales, pero podemos limitar la cantidad de decimales añadiendo **.x** después del %, donde x indica **cuántos decimales queremos mostrar**.

```
double precio=1.5;
String precioS=String.format("Precio con 2 decimales: %.2f",precio);
System.out.println(precioS); //Precio con 2 decimales: 1.50

double cantidad=2.2356;
String cantidadS=String.format("Cantidad con 3 decimales: %.3f",cantidad);
System.out.println(cantidadS); //Cantidad con 3 decimales: 2.236
```


Clase String. Formatear cadenas

Ejemplo

```
String producto = "Café";  
double precio = 1.5;  
int cantidad = 3;  
  
String ticket = String.format("Producto: %s \nPrecio: %.2f euros,\nCantidad: %d \nTotal: %.2f euros",  
                             producto, precio, cantidad, precio * cantidad);  
System.out.println(ticket);
```

Resultado:

```
Producto: Café  
Precio: 1.50 euros,  
Cantidad: 3  
Total: 4.50 euros
```

Clase String

- **String replace (String , String)** Reemplaza todas las ocurrencias de una subcadena por otra. También se puede usar **replaceAll** o **replaceFirst**. La primera se comporta igual pero además de recibir una cadena puede recibir una expresión regular. La segunda, también puede recibir expresiones regulares, y solo cambia la primera ocurrencia de la subcadena.
- **String[] split (String regex)** Devuelve un array de String resultado de partir la cadena usando como separador el argumento (cadena o expresión regular).

Ej: `"hola como estás".split(" ")` → `new String[] {"hola", "como", "estás"}`

Clase String

Los métodos no cambian el valor de la cadena (si hacemos por ejemplo un, *cadena.toLowerCase()*, la cadena original no cambia).

En *replaceAll*, *replaceFirst* y los siguientes caracteres no se pueden reemplazar directamente: \$ ^ . * + ? [](). Hay que poner \\? y similares, (la razón es que no se busca una cadena sino una expresión regular)